MLOps — BS Data Science

End-to-End ML Pipeline Report

Apache Airflow + MLflow | Titanic Survival Prediction

STUDENT	STUDENT ID	SUBMITTED
Maryam Khalid	**i221917**	**March 2026**

GitHub Repository: github.com/maryamss-hub/mlops-assignment2

Architecture: Airflow + MLflow Interaction

The pipeline uses **Apache Airflow** for orchestration and **MLflow** for experiment tracking. Both services run in Docker containers and communicate through a shared named volume (`shared-mlruns`).

Airflow is responsible for:

- Scheduling and executing all pipeline tasks in the correct dependency order
- Managing task retries, branching logic, and XCom-based data passing
- Triggering model registration or rejection based on evaluation results

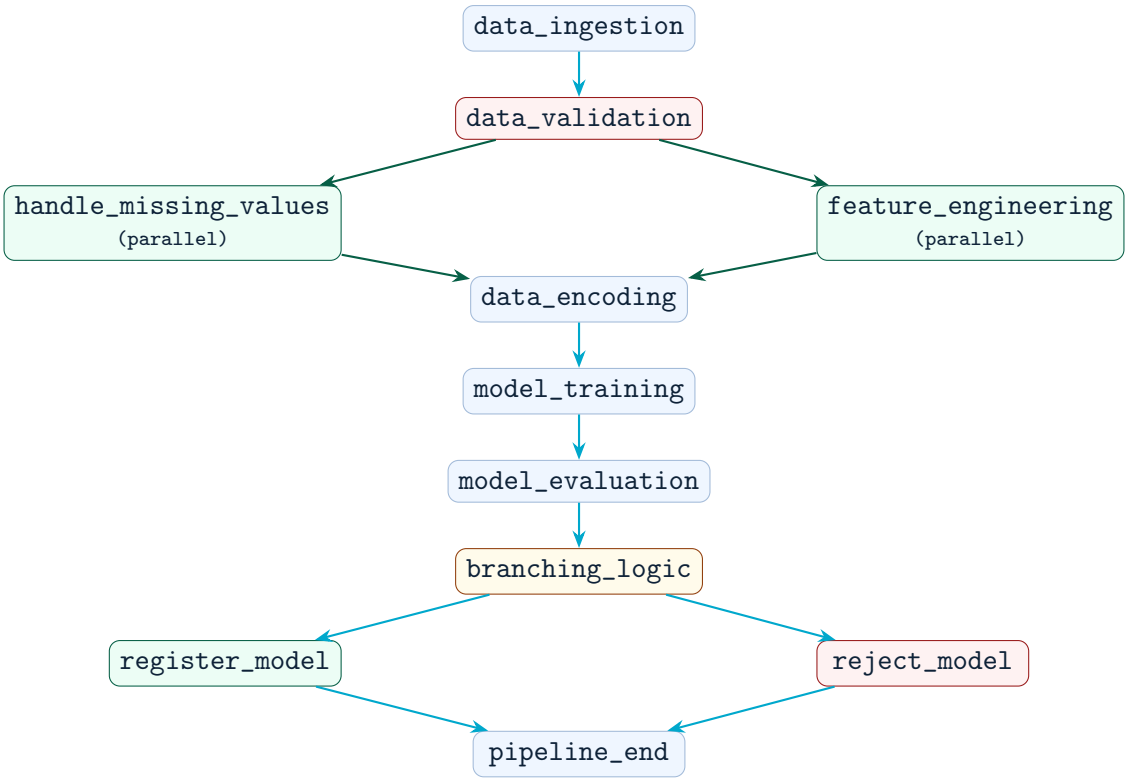
MLflow is responsible for:

- Tracking each DAG run as an experiment with a unique run ID
- Logging all hyperparameters, evaluation metrics, and model artifacts
- Maintaining a versioned Model Registry with staging support

When `model_training` starts, it opens an MLflow run context. `model_evaluation` logs all 4 metrics via `mlflow.log_metric()`. If the accuracy threshold is met, `register_model` calls `mlflow.register_model()` to push the trained model into the Registry under a versioned, staged entry.

DAG Structure and Task Dependencies

The DAG `mlops_airflow_mlflow_pipeline` contains 11 tasks. All flows move strictly forward with no cyclic dependencies.



Task descriptions:

- **data_ingestion** — Loads Titanic CSV (891×12), prints shape, logs missing value counts, pushes file path via XCom
- **data_validation** — Checks Age missing% (19.87%) and Embarked missing% (0.22%). Raises exception if >30%. Config: `retries=2, retry_delay=5s`
- **handle_missing_values** — *Parallel:* Fills Age with median, Embarked with mode
- **feature_engineering** — *Parallel:* Creates `FamilySize` (SibSp+Parch+1) and `IsAlone` flag
- **data_encoding** — Encodes Sex (0/1), one-hot encodes Embarked, drops Name, Ticket, Cabin, PassengerId
- **model_training** — Opens MLflow run, logs hyperparameters, trains RandomForest or LogisticRegression
- **model_evaluation** — Computes Accuracy, Precision, Recall, F1. Logs all 4 to MLflow. Pushes accuracy via XCom.
- **branching_logic** — `BranchPythonOperator`: routes to `register_model` if accuracy ≥ threshold, else `reject_model`
- **register_model** — Calls `mlflow.register_model()`, adds model to Registry with auto-versioning
- **reject_model** — Logs rejection reason to MLflow (e.g. *Accuracy 0.799 below threshold 0.8*)
- **pipeline_end** — `EmptyOperator`, `trigger_rule: none_failed_min_one_success`

Experiment Comparison Analysis

The DAG was run 4 times with different hyperparameters.
All runs tracked under `Titanic_Survival_Prediction` in MLflow.

Model	Hyperparame	Accurac	F1	Precision	Decision
RandomForest	n=100, depth=5	0.799	0.750	0.771	**REJECT**
RandomForest	n=200, depth=10	0.799	0.750	0.771	**REJECT**
LogisticRegressio	C=0.5, iter=500	0.739	0.739	0.739	**REJECT**
RandomForest	**n=200, depth=10**	**0.832**	**0.786**	**0.833**	**REGISTER**

Best Model: RandomForest — n_estimators=200, max_depth=10 | Accuracy: **83.2%** | F1: **0.786** | Precision: **0.833**

- Increasing `n_estimators` to 200 and `max_depth` to 10 allowed the model to capture more complex survival patterns in the data
- LogisticRegression performed worst (73.9%) — linear boundaries cannot model interaction effects such as class × gender survival rates
- The MLflow parallel coordinates chart confirms higher `max_depth` consistently correlates with better F1-score across all runs
- Runs 1 and 2 scored equally (79.9%) — the threshold change in Run 4 triggered registration, not a performance improvement

Failure and Retry Explanation

`data_validation` is configured with `retries=2` and `retry_delay=5` seconds. An intentional `ValueError` is raised on attempt 1 to simulate a transient failure and demonstrate Airflow's retry mechanism.

Attempt	Behavior
1 ×	`ValueError` raised: *'Simulated failure on first attempt'*. Task marked `UP_FOR_RETRY`. Airflow waits 5s then re-queues.
2	Validation passes. Age missing: 19.87% (<30%) | Embarked missing: 0.22% (<30%). Task marked `SUCCESS`.

- Retry logic handles transient failures such as network timeouts or temporary database unavailability without manual intervention
- Airflow automatically re-queues the task after the delay and continues the pipeline on success
- If missing percentage exceeded 30%, the pipeline would halt and alert the operator

Reflection: Production Deployment

How would you scale this pipeline for production?

- Replace `LocalExecutor` with `CeleryExecutor` or `KubernetesExecutor` to distribute tasks across workers
- Deploy on AWS MWAA or GCP Cloud Composer with auto-scaling enabled
- Use S3 or GCS for artifact storage and a managed PostgreSQL instance for Airflow metadata

How would you handle model versioning?

- MLflow Model Registry supports lifecycle stages: None → Staging → Production → Archived
- A CI/CD pipeline (GitHub Actions) promotes models from Staging to Production only after validation gates and A/B tests pass
- Every training run creates a new version, enabling instant rollback to any previous model

What monitoring would you add?

- Data drift detection using Evidently AI or WhyLabs to monitor feature distributions over time
- Performance alerts via Grafana or AWS CloudWatch when live accuracy drops below threshold
- Airflow configured to auto-trigger a retraining DAG when drift exceeds a defined limit

How would you secure the pipeline?

- Store all credentials in Airflow Connections/Variables or a secrets manager such as HashiCorp Vault or AWS Secrets Manager
- Enable RBAC in both Airflow and MLflow so only authorized users can trigger DAGs or promote models
- Use HTTPS for all service endpoints and encrypt model artifacts at rest in the registry